🛡️ Eunomia Reviewer Spec Doc

🔗 Please always check the updates on these materials, and check Outlier Community for recent announcements!

Regardless of the score you give to the attempt, as a reviewer, you must always fix the task and make sure it is perfect! Your revised tasks will always move forward. No send back to queue allowed.

- Attempter Instructions
- QC Spec Doc
- Common Errors
- Reviewer Instruction

🗒️ Grading Checklist

Rating	Action	Criteria (Checklist)
1 – Terrible	**Critical Errors**	🚨 Ratability Fail: Marked ratable but is non-English, lacks context (missing files/history), niche platform, or is a general question unrelated to code. 🚨 Effort Fail: Attempter put little to no effort into rubrics or justifications. 🚨 Golden Response Fail: The rewrite is broken, contains syntax errors or code bug, or does NOT pass All the criteria.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z

2 – Bad	**Major Errors**	🟠 Counterproductive Rubrics: At least one criterion penalizes good responses / rewards bad responses such that the inclusion of this criteria would make the response objectively worse. 🟠 Overly Prescriptive Rubrics: At least one criterion is overly prescriptive, that it fails technically sound, industry-standard solutions. 🟠 Redundancy: Two or more rubrics penalize the exact same error. 🟠 Golden Response: Rewrite fails to pass even one of the Mandatory rubrics created by the attempter. 🟠 Rubric Accuracy: Criterion contains a factual error or contradicts the prompt. 🟠Missing Coverage: The rubric is missing at least one criteria that checks for an **explicit** requirement in the prompt (only flag explicitly written requirements here)
3 – Okay	**Noticeable Issues**	🟡 Scoring Rule Violation: Failed 1 Code/IF rubric or 30-50% of QoL rubrics (should be a Score 3). 🟡 Don't spend too much time on editing those Nice to Have QoL, and spend more time on writing **mandatory Instruction Following (IF) rubrics** that can differentiate (Fail) models. 🟡 Don't fail an attempt if there is not enough QoL (less than 40%), give it a 3 is fine. 🟡 QoL Rubrics: QoL rubric has All Pass for the 4 models, failed to differentiate model responses enough. (don't fail such attempts, but score it a 3) 🟡 Vague/Generic Rubrics: Non-QoL rubrics lack "Technical Teeth" (e.g., "Code is readable"). 🟡 Reiteration: Rubric simply repeats the prompt without defining a success standard. 🟡 Justification: The explanation is generic, one-sentence, or doesn't reference specific rubric failures.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z

4 – Good	**Minor Issues Only**	🟢 Formatting/Tone: Minor issues in the rewrite or justification (e.g., conversational "fluff"). 🟢 Rubric Nuance: 0-20% of weights (Mandatory vs. Nice-to-Have) are slightly off but don't flip the overall score. 🟢 QoL Failures: Response failed <30% of QoL rubrics (should be a Score 4).
5 – Excellent	**No Issues**	✅ Perfect Task: Zero errors. Rubrics are atomic, self-contained, and provide the ground truth. ✅ MECE: Rubrics are Mutually Exclusive and Collectively Exhaustive (~10 total). ✅ Technical Teeth: Rubrics use "LIKE/e.g." to define valid technical paths.

⚖️ Finding the Balance: Prescriptive vs. Generic

🛑 1. Avoid: Overly Prescriptive (Rigid) Rubrics

The Test: If a model provides a different but technically sound solution (different library/pattern), would this rubric force a failure?

- **Why it's bad:** It penalizes "correct" models for not being mind-readers.
- **Exceptions:** Only be prescriptive if the prompt explicitly demands a method (e.g., "Use a `for` loop") or for security (e.g., "Use parameterized queries").

☁️ 2. Avoid: Overly Vague (Generic) Rubrics

The Test: If you swapped this Python prompt for a Java prompt, would the rubric still work?

- **Why it's bad:** "Code is clean" or "Provides a summary" are filler. They don't evaluate the model's coding logic.

💡 The "Technical Teeth" Strategy

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z

- **Use "LIKE / e.g.":** Give the judge a standard while remaining flexible.
  - *Bad:* "Uses initializer injection."
  - *Good:* "Uses a standard form of Dependency Injection (e.g., Initializer, Property, or Method injection)."
- **Provide Ground Truth:** Since the LLM Judge doesn't "know" the answer, you must define it.
  - *Bad:* "Decodes the file correctly."
  - *Good:* "Identifies the compression algorithm (LZ-String) and method (decompressFromBase64) to deserialize the format."

🛠️ Common Reviewer Checkpoints

Feature	❌ Bad (Failing)	✅ Better (Passing)
Golden Response	Has syntax errors, or Fails any of the rubrics or includes "fluff" like "I'd be happy to help!"	• **Rewrite the golden response until it passes ALL rubrics.** • Golden Responses must be professional and direct. Delete "filler" praise such as "Great question!", "Great use case!", or "I'd be happy to help!". • Contextual Accuracy: Avoid tone-deaf affirmations. For example, if a user shows an error, the response should not say "Exactly!". You should remove those!
Atomicity	"The code is efficient AND has comments."	Split into two rubrics.
Overly Prescriptive	"The code uses a for loop to iterate through the array." (Forces one method)	"The code iterates through the array using an **efficient method (e.g., for loop, map, or filter)**."
Self-Contained	"Response identifies the first president."	"Response identifies the first president as George Washington."

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z

Coverage	Prompt asks for a docstring but no rubric checks for it.	Create a Mandatory rubric for every **explicit** prompt constraint.
Redundancy	R1: "Proper imports." R2: "No unimported modules."	Combine into one: "All required modules (e.g., pandas) are imported."
Ratability	Guessing what the user means by "the template" when no file is attached.	Mark as **Unratable** due to missing context.
Technical Teeth / IF Depth	Spending high effort on generic QoL rubrics (e.g., "formatting") for simple, easy prompts.	Adding **Instruction Following (IF) rubrics** to force mandatory failures and increase technical difficulty.
Rubric Scaling & Complexity	Including an excessive number of rubrics for low-complexity prompts (e.g. a 13 rubric set for a prompt like “What is 1<<n in cpp”)	Remove unnecessary rubrics, crafting a set that completely covers the prompt with Mutually Exclusive and Collectively Exhaustive criteria.

✍️ Constructive Feedback Template:

Overall Task Quality:
A brief summary of the annotation quality (e.g., accurate, partially correct, needs revision).

Strengths:

- Highlight what the attempter did well (accuracy, formatting, effort, etc.).

Areas to Improve:

- *Area 1:* (Clear description of the issue)

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z

- *Area 2:*
- *Area 3:*

  (Use bullet points to make it easy to read and action-oriented.)

Corrected Example:
Provide a corrected version, improved annotation, or demonstration of how the task should look when done correctly.

Next Steps:
Actionable guidance for how the annotator can avoid similar issues in the future.

Conclusion:
A short, encouraging but honest closing line reaffirming the importance of following the guideline.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.858Z